GUIA VISUAL 074

# SUBSHELLS, AGRUPAMENTO E HERE-DOCUMENTS

*O script de implantação tinha uma função com `cd "src/$1"` e `make`. O módulo **rede** compilou; **parser** e **interface** falharam com "diretório não encontrado". O `cd` dentro da função mudou o diretório do **script inteiro**, e a segunda chamada tentou entrar em `src/rede/src/parser`. A correção cabe em **dois caracteres**: `( cd "src/$1" && make )`. Saber o que atravessa a fronteira de um subshell — e em que direção — separa quem escreve shell por hábito de quem escreve com intenção.*

**074**
GUIAS LINUX

## A FRONTEIRA DO SUBSHELL · E O AGRUPAMENTO QUE NÃO TEM FRONTEIRA NENHUMA

**SHELL PAI**
```
PWD=/projeto
contador=0
umask 022
trap limpar EXIT
opções: set -e
```

herda tudo na entrada →
← só o código de saída volta

**SUBSHELL ( ... )**
recebe **cópia** de tudo
`cd /tmp` → só aqui dentro
`contador=99` → só aqui dentro
`exit 1` → encerra só o subshell
traps de sinal voltam ao padrão

**( COMANDO1; COMANDO2 ) — ISOLA**
Use para **isolar cd**, mudar `umask` ou opção do shell temporariamente, ou rodar algo em paralelo com &. Cada um é um **fork**: custa.

**{ COMANDO1; COMANDO2; } — AGRUPA**
Mesmo processo: cd e atribuição **persistem** e `exit` derruba o script. Exige **espaço depois do** { e **; antes do** } — são palavras reservadas, não operadores.

**Subshells nascem sem que você escreva parênteses:** em **cada estágio** de um pipeline (`a | b | c` são três), em toda substituição de comando `$(…)`, em todo comando posto em segundo plano com **&** e em toda substituição de processo `<(…)` / `>(…)`. É a explicação de `n=0; seq 3 | while read x; do ((n++)); done; echo $n` imprimir `0`: o `while` rodou num processo que já morreu levando o `n` junto. **O uso legítimo do { } é aplicar um redirecionamento, um | ou um && a vários comandos de uma vez** — sem ele seria preciso repetir `>> relatorio.txt` em cada linha, e a primeira teria de ser **>** para truncar, exatamente o tipo de assimetria que se erra. **E antes de abrir um subshell, verifique se existe uma expansão que faz o mesmo:** `$(<arquivo)` é implementado internamente pelo Bash e não cria processo nenhum (contra dois em `$(cat arquivo)`); `${f%.log}` substitui `$(basename "$f" .log)`; `wc -l < "$f"` substitui `cat "$f" | wc -l`. Num laço de dez mil iterações com três substituições cada, são **trinta mil processos** de diferença.

## HERE-DOCUMENTS E HERE-STRINGS: AS QUATRO VARIANTES

**<<EOF** — EXPANDE
**expande $var, $(cmd), **
Quando o bloco precisa de **valores do script**. Cuidado: um **$** literal no texto vira expansão sem aviso.

**<<'EOF'** — LITERAL
**não expande nada**
**O padrão** para gerar script, configuração e código. As aspas no delimitador são a única diferença visível.

**<<-EOF** — INDENTÁVEL
**remove as tabulações iniciais**
Permite indentar o bloco dentro de um **if**. Só **TAB**, nunca espaço — editor que converte TAB quebra em silêncio.

**<<< "texto"** — LINHA ÚNICA
**here-string, sem pipe**
`read -r a b <<< "$linha"` substitui `echo | read` e **evita o subshell**: as variáveis sobrevivem.

kali@kali: ~/lab-subshell

```
# redirecionamento aplicado ao GRUPO: o uso legítimo do { }
$ { echo "=== $(date +%F) ==="; df -h; echo "=== fim ==="; } > rel.txt
# gerar arquivo: aspas no delimitador, SEMPRE, por padrão
$ cat > /usr/local/bin/limpar.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
for f in /var/tmp/*; do rm -rf -- "$f"; done
EOF
# sem as aspas, $f expandiria AGORA: o disco receberia rm -rf -- ""
# escrever onde exige privilégio — sudo echo > NÃO funciona
$ sudo tee /etc/systemd/system/coleta.service >/dev/null <<'EOF'
[Service]
Type=oneshot
ExecStart=/usr/local/bin/coletar.sh
EOF
# here-string: sem pipe, sem subshell — o read sobrevive
$ IFS=: read -r usuario _ uid _ <<< "kali:x:1000:1000"
$ echo "$usuario $uid"
kali 1000
# isolar o ambiente de UMA invocação
$ ( export HTTP_PROXY=http://127.0.0.1:8080; ferramenta )
```

### AQUI OU LÁ? O HEREDOC SOBRE SSH

| | |
|---|---|
| `<<EOF` | o bloco é expandido **localmente**: `$PARTICAO` chega já resolvido, e só o que você escapar com `\$` sobra para o remoto |
| `<<'EOF'` | o bloco vai **literal**: `$HOSTNAME` é resolvido **no remoto** |
| `teste antes` | troque `ssh servidor` por `cat` e **olhe o bloco** — um `$(…)` que você achou que rodaria aqui e roda lá é execução não pretendida na máquina de outra pessoa |
| `ssh -n` | em laço, o **ssh engole o resto da entrada** e o laço acaba na primeira volta. `ssh -n` ou `< /dev/null` |

### DETALHES QUE TRAVAM O SCRIPT

| | |
|---|---|
| `delimitador` | sozinho na linha, **sem espaço antes nem depois**. Um espaço invisível depois do fechamento é a causa nº 1 de **unexpected end of file**: o shell lê o resto do arquivo procurando um fim que não chega |
| `<<-'EOF'` | existe e junta as duas coisas — **literal e indentável**. É a forma de escolha para gerar arquivo de dentro de um bloco aninhado |
| `em pipeline` | o heredoc alimenta o **primeiro** comando: `sort <<'EOF' \| uniq -c` |
| `coproc` | subshell com quem se conversa, dois pipes já ligados. Nicho: mais que uma troca de linhas **trava em impasse** quando o buffer enche |

**Gerar script com `<<EOF` sem aspas grava o comando destrutivo já expandido no disco.** No bloco `for f in /var/tmp/*; do rm -rf -- "$f"; done`, se o delimitador não estiver entre aspas o `$f` é resolvido **na hora de gerar o arquivo** — e como `f` não existe no script gerador, o que fica gravado é `rm -rf -- ""`. O defeito só aparece quando o script gerado roda, com o privilégio que ele tiver